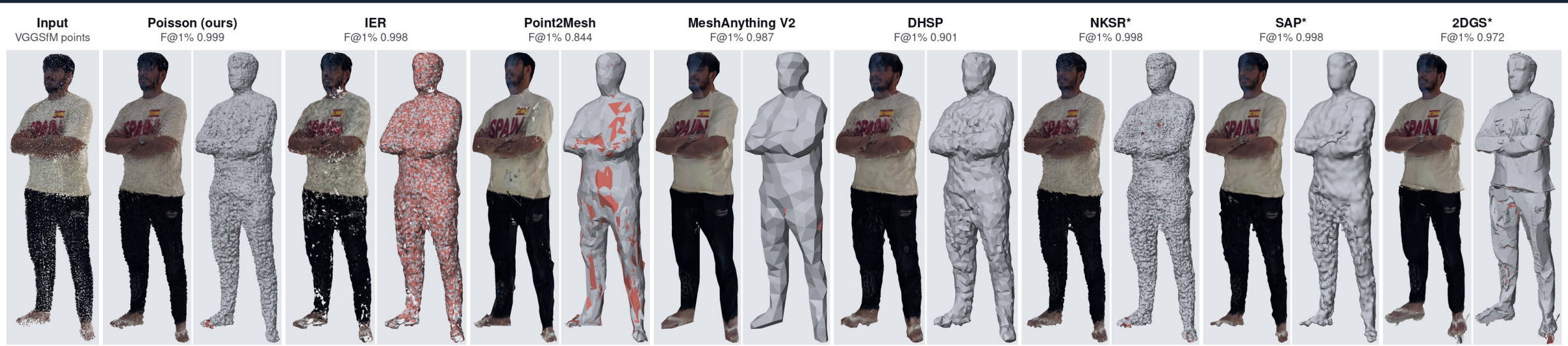
CV802 ADVANCED 3D COMPUTER VISION · SUB-PROJECT 2: SURFACE EXTRACTION

From Coloured Points to Printable Textured Meshes

Eight surface-reconstruction methods, one shared pipeline, our own screened Poisson and photo texturing, and a mesh viewer

Anas Khan · MBZUAI



The task

Week-5 brief (slides 5-7): 3D points with colour -> textured mesh (.obj), bonus for a mesh viewer



input: 138,677 coloured points → output: textured .obj

Method on slide 7	What we did	Colour
Poisson (DO NOT USE API)	own implementation, GPU	our texturing
IER (ECCV'20)	official code + weights	our texturing
Point2Mesh (SIGGRAPH'20)	official code	our texturing
MeshAnything (2024)	official V1 / V2 + weights	our texturing
DHSP (ICCV'21)	official code, full pipeline	own (2D prior)
Your own novel methods	photo texturing, DHSP fix, viewer	-
Extra (beyond slides)	NKSR, SAP, 2DGS	our texturing

- **Deliverables (slide 8):** presentation · code · one-page document
- **Bonus:** browser mesh viewer, side-by-side, CPU only
- **Our goal:** best quality, 3D-printable

Three input clouds of the same person

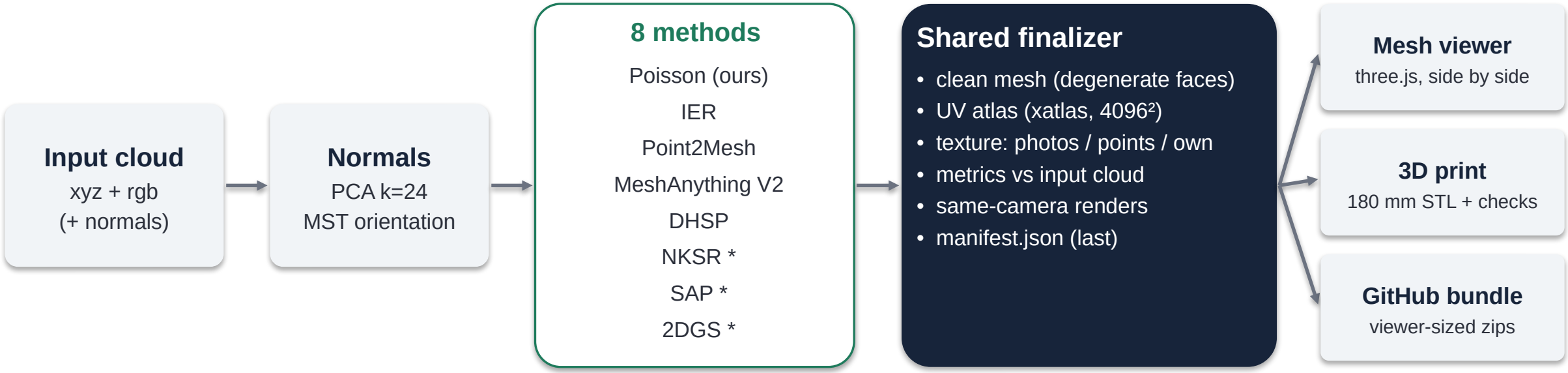
All from Sub-project 1; every method runs on all three



Input	Points	Normals	Character
VGGSfM (main)	138,677	ours (PCA + MST)	clean, sparser, mask-cleaned
COLMAP MVS	487,451	COLMAP PatchMatch	dense; streaks + floating outliers
MVS-clean	487,047	COLMAP PatchMatch	404 floaters removed (clean_cloud.py)

One pipeline for every method

Methods only produce geometry; everything after it is shared and identical



- **Contract:** run.sh <run_dir> → geometry.ply + method_info.json + progress.json; finalize.sh <run_dir> → mesh.obj / .mtl / texture, metrics.json, previews, manifest.json
- **Compute:** one RTX 5000 Ada (32 GB) through Slurm; restartable job queue in tmux with live GPU monitor; nothing runs on the login node
- **Scale:** ≈110 finished runs over 3 inputs (ablations, seeds, texture variants); best run per method selected automatically (select_best.py)

Screened Poisson, implemented from scratch

No Poisson / marching-cubes library: normals, solver and mesh extraction are our own code (PyTorch tensors on the GPU)

1 Oriented normals

PCA (k = 24) gives ±n; signs propagated along a minimum spanning tree, then each component flipped outward (local-centroid vote)

2 Splat

inward normals × local spacing² → vector field V on a 384^3 grid ([1,2,1] smoothing ≈ B-spline)

3 Solve

screened Poisson system, Jacobi-preconditioned CG on the GPU, coarse-to-fine 96^3 → 192^3 → 384^3

4 Extract

iso = weighted mean of χ at the samples; own marching tetrahedra (Kuhn split, welded vertices → crack-free, consistently oriented)

5 Trim

drop faces > 2 voxels from every sample, keep large components (watertight variant: no trim)

$$(-\Delta + \beta S^\top W S)\chi = -\nabla \cdot \mathbf{V} + \beta S^\top W c$$

- χ: indicator (1 inside, 0 outside); $\nabla\chi \approx V$ (normals)
- S: trilinear sampling of χ at the points; c: iso-value
- β (α = 4): screening weight; α = 0 → classic 2006 Poisson
- MST edge cost:

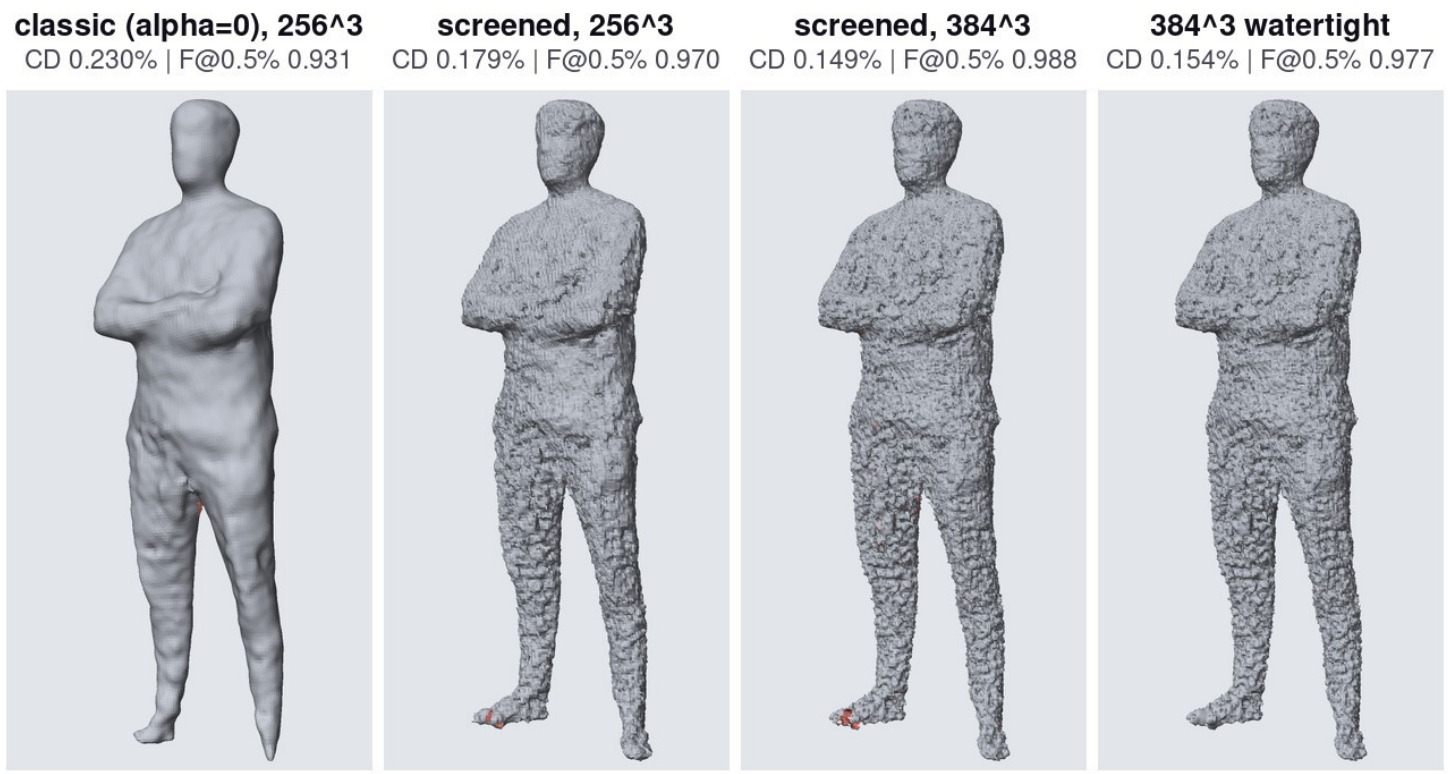
$$w_{ij} = 1 - |\mathbf{n}_i \cdot \mathbf{n}_j|$$

Run (VGGSfM)	Faces	CD %	F@1%	Time
384^3, screened, trimmed	1.02M	0.149	0.999	39s
MVS-clean, 384^3	669k	0.101	0.999	72s

- **Deviation from the paper:** dense grid + cascadic CG instead of an adaptive octree + multigrid; marching tetrahedra instead of marching cubes
- **Sanity check:** sphere radius error 0.0007 (selftest.py)

Poisson: what each design choice buys

Ablation on the VGGSfM cloud (shaded geometry)



Variant	CD %	F@0.5%	NC	Faces
classic, 256^3	0.230	0.931	**0.84**	265k
screened, 256^3	0.179	0.970	0.79	358k
screened, 384^3	**0.149**	**0.988**	0.75	1.02M
384^3 watertight	0.154	0.977	0.75	1.05M

- **Screening** pins χ to the samples → +0.04 F@0.5%
- **Finer grid** → more detail, also more scan noise
- **No trimming** → closed surface for printing at almost no cost
- **NC** rewards smoothness, CD/F reward closeness: read them together

Learned priors: IER and MeshAnything

Pretrained networks only work inside their training domain

IER meshing (ECCV'20)

- kNN candidate triangles → pretrained network scores each by
- ≈1 on the surface, ⩾1 across gaps (arm ↔ chest) → greedy manifold selection

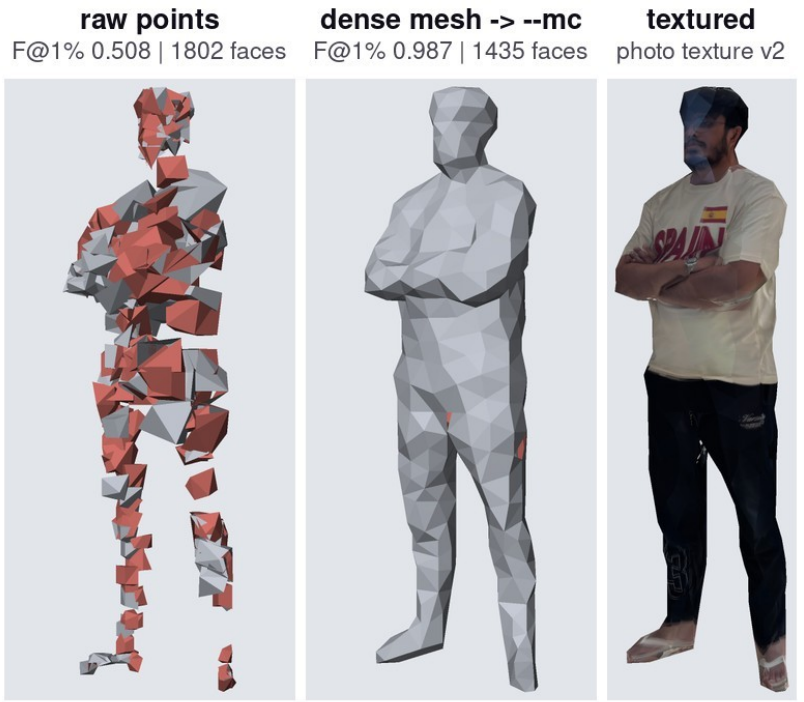
$$\mathrm{IER}(p,q) = \frac{d_{\mathrm{geodesic}}(p,q)}{\|p-q\|}$$

- **Our fix:** FPS to the training density (12.5k pts, unit diagonal) → F@1% 0.998
- **Caveat:** inconsistent winding (red faces), holes → two-sided photo texturing

MeshAnything run	F@1%	faces
V1, raw points	0.442	795
V2, raw points (greedy)	0.113	624
V2, raw points (sampled)	0.508	1802
V1, mesh --mc	0.929	471
V2, mesh --mc 7	**0.987**	1435
V2, mesh --mc 8	0.972	1810

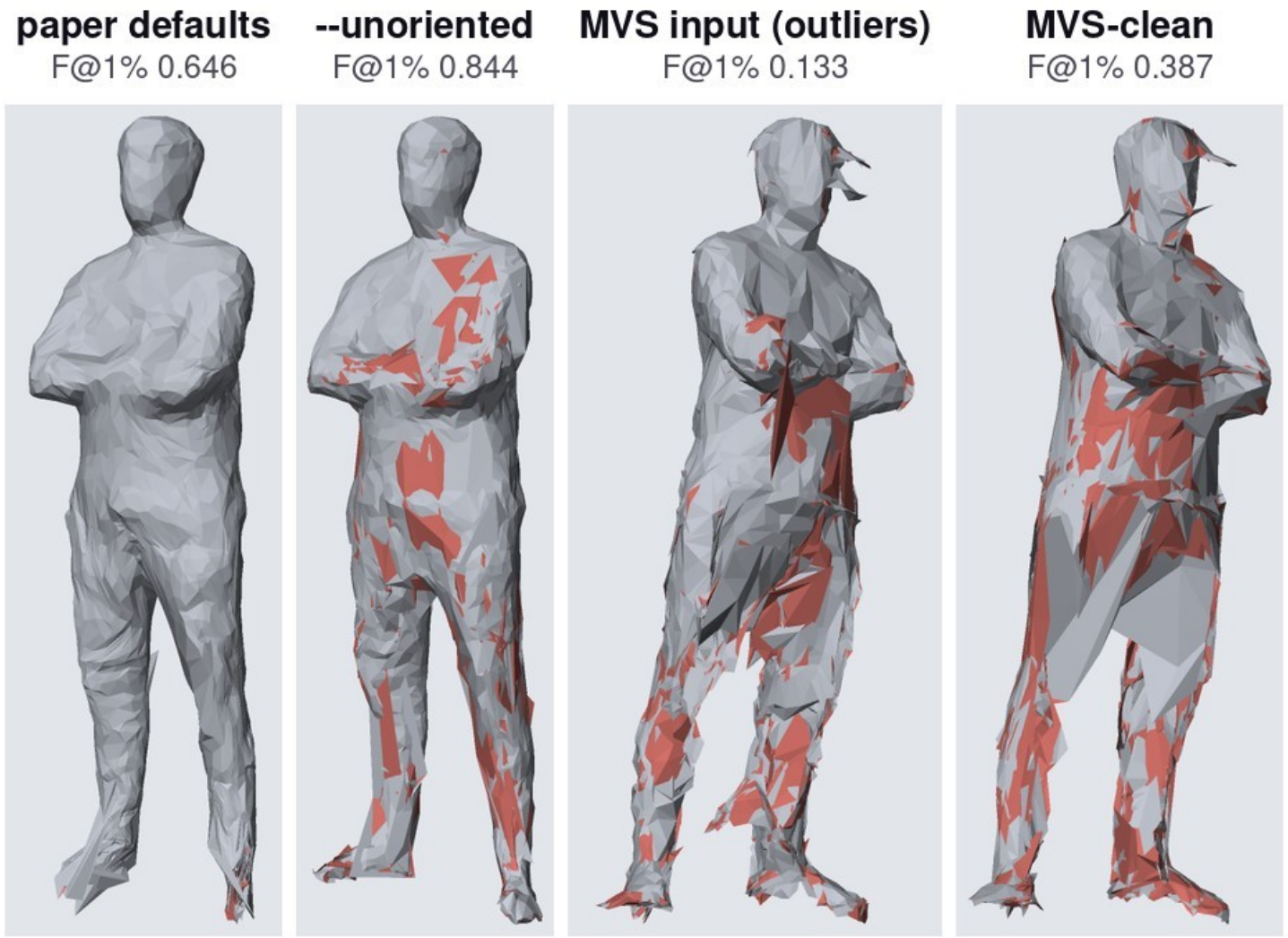
MeshAnything V2 (2024)

- autoregressive transformer generates an artist mesh face by face (≤1.6k faces)
- raw points: F@1% ≤ 0.51 → our Poisson mesh via the authors' --mc path: **0.987**



Self-prior: Point2Mesh

A randomly initialised CNN, optimised on this one cloud, deforms a convex hull

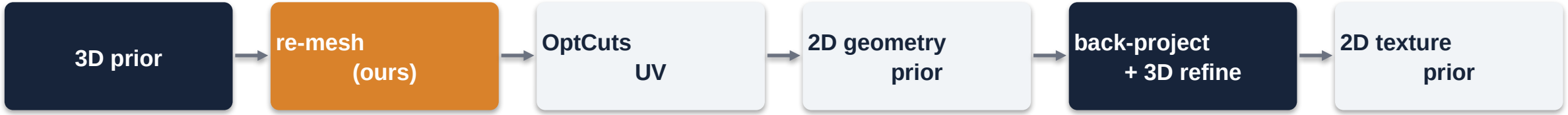


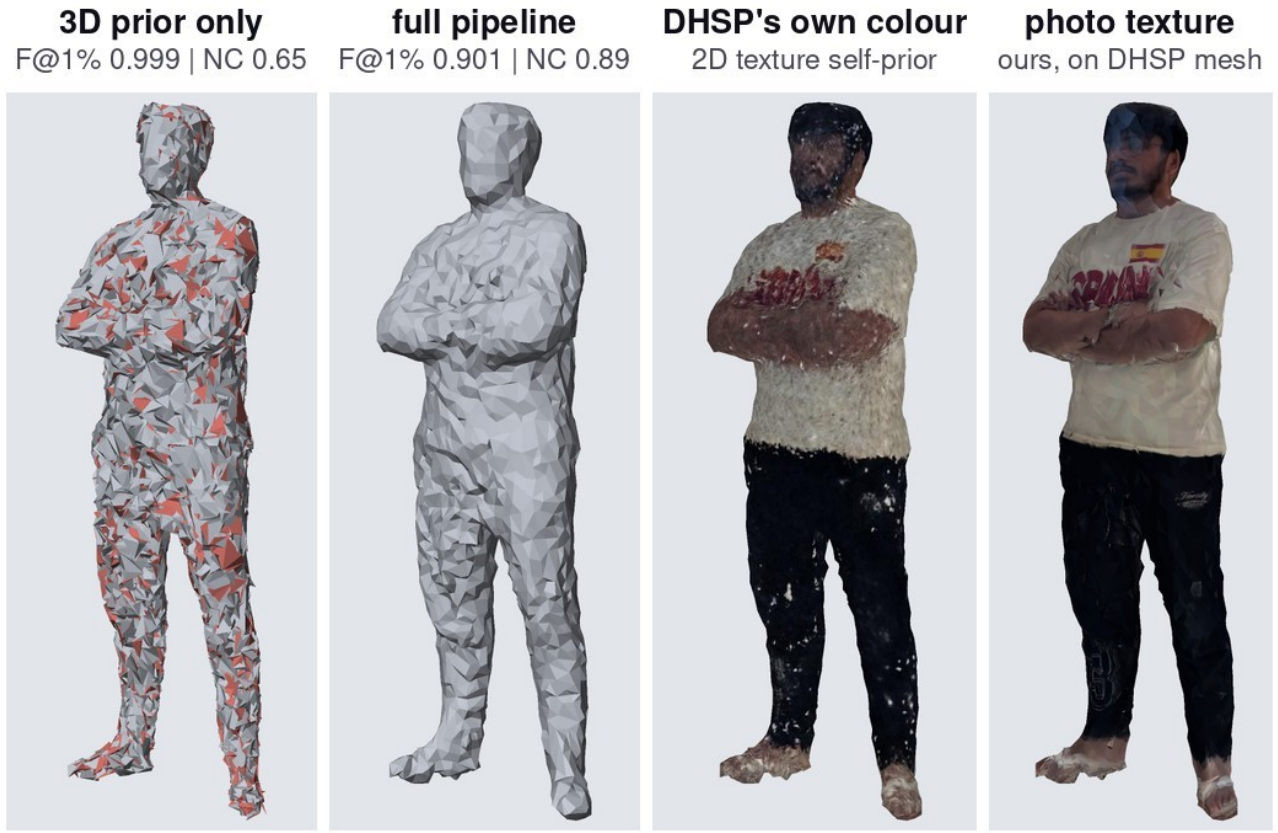
Run	Input	F@1%	CD %
paper defaults	VGGSfM	0.646	0.82
--unoriented	VGGSfM	**0.844**	**0.55**
--unoriented	MVS	0.133	3.98
--unoriented	MVS-clean	0.387	1.30

- loss: Chamfer(mesh, points) + normal term; mesh = hull + CNN(θ) displacements
- hull → Manifold (2k faces) → CNN displacements → subdivide + re-manifold
- **Normal orientation** matters: unoriented loss +0.20 F
- **Outliers** break shrink-wrapping (MVS)
- **Watertight** by construction → printable

DHSP: getting the full hybrid pipeline to run

Deep Hybrid Self-Prior (ICCV'21): 3D mesh prior + 2D image priors, recovers shape and colour



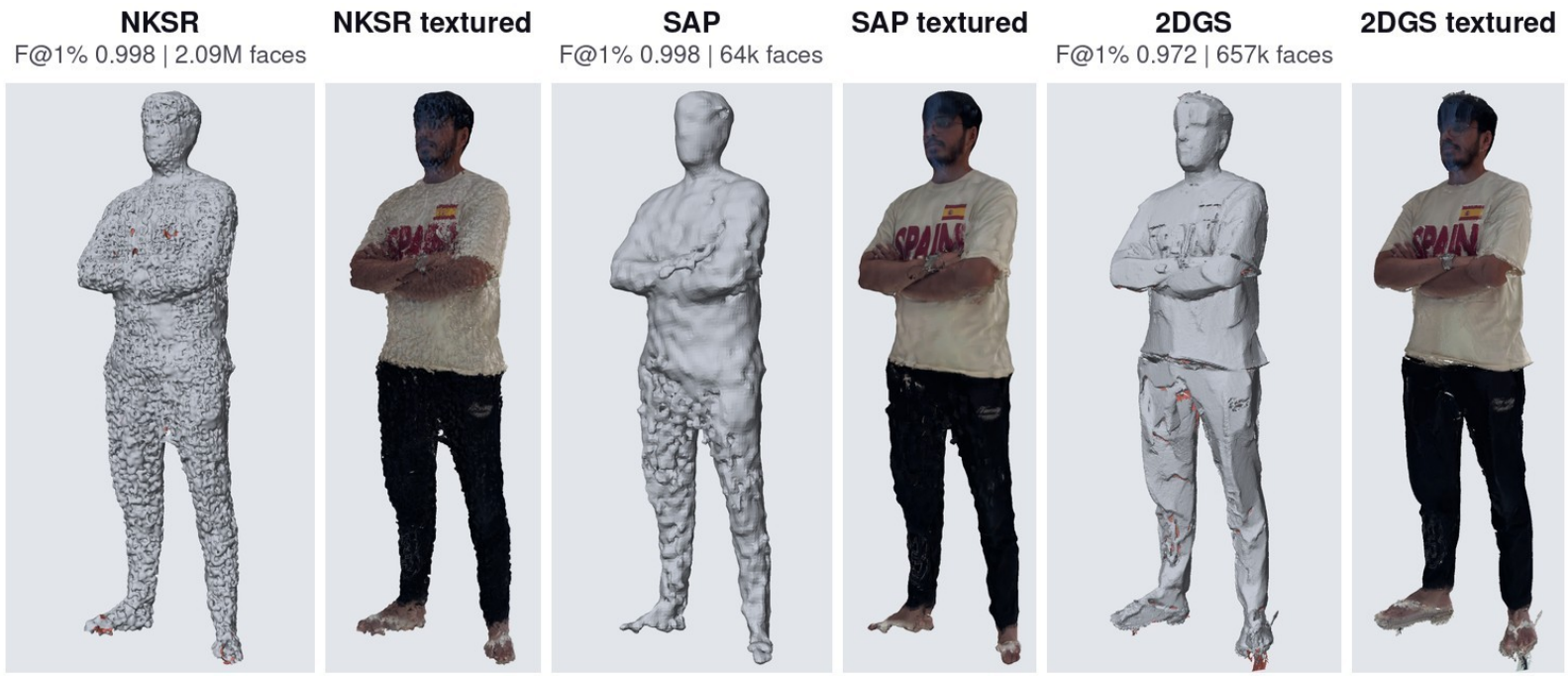


DHSP run (VGGSfM)	F@1%	NC	WT
v1 3D prior only	0.999	0.65	yes
v2 + xatlas 2D texture	0.999	0.65	no
v4 re-mesh + OptCuts	0.874	**0.89**	yes
v5 + Laplacian 1	0.901	**0.89**	yes

- **Crumpled prior:** F-score high, surface folded (NC 0.65)
- **OptCuts:** no convergence in 15 h → converges in seconds after re-meshing
- **MVS / MVS-clean:** F@1% 0.900 / 0.916 — below the 3D prior (0.98): MVS outliers become fins; final atlas by xatlas (genus-2 surface)

Beyond the list: three recent methods

Added to see where current methods stand on the same input



NKSR

best on dense MVS (CD 0.08%), seconds; open, non-manifold

Shape As Points

optimises oriented points through a spectral Poisson solver ($32^3 \rightarrow 256^3$); watertight

2D Gaussian Splatting

surfels fitted to the masked photos, TSDF fusion of rendered depth; sharpest appearance, uses images rather than only points

Method	Idea	F@1%	Time	WT
NKSR (CVPR'23)	pretrained kernel field	0.998	7s	no
SAP (NeurIPS'21)	differentiable Poisson	0.998	8min	yes
2DGS (SIGGRAPH'24)	surfels fit to photos	0.972	47min	no

Texture: projecting the photos onto the mesh

Our own multi-view texturing for every shape-only method



Per texel, per photo:

- in front of the camera, inside the image
- inside the person mask
- faces the camera (cos θ > 0.25)
- passes the z-buffer test (no occluded colour)

Blend:

- frontal + close views win; ^8 keeps it sharp
- per-photo exposure gain vs point colours
- 4096² texture; unseen texels → point colour
- DHSP keeps its own colour; NKSR/2DGS also get photos

$$c(t) = \frac{\sum_k w_k^8 \, g_k \, I_k(\pi_k(\mathbf{x}_t))}{\sum_k w_k^8}, \qquad w_k = \cos^4 \theta_k \left(\frac{d}{z_k}\right)^2$$

I_k: photo k, π_k: its COLMAP projection, θ_k: angle normal ↔ view, z_k: depth, g_k: exposure gain, d: scene diagonal

Results: best run of every method on all three inputs

Measured against each input cloud (no ground-truth surface exists)

		VGGSfM (main)			MVS		MVS-clean		
Method	**Faces**	**CD % ↓**	**F@1% ↑**	**NC ↑**	**CD % ↓**	**F@1% ↑**	**CD % ↓**	**F@1% ↑**	**WT**
Poisson (ours)	1.02M	**0.149**	**0.999**	0.75	0.127	0.999	0.127	0.999	
IER	17k	0.187	0.998	0.82	0.120	**1.000**	0.123	**1.000**	
Point2Mesh	10k	0.553	0.844	0.80	3.977	0.133	1.298	0.387	✓
MeshAnything V2	1.4k	0.295	0.987	0.82	0.518	0.873	0.438	0.908	
DHSP	6.0k	0.583	0.901	**0.89**	0.636	0.900	0.627	0.916	✓
NKSR *	2.09M	0.156	0.998	0.75	**0.080**	**1.000**	**0.080**	**1.000**	
SAP *	64k	0.195	0.998	0.81	0.197	0.963	0.239	0.948	✓
2DGS *	657k	0.289	0.972	0.75	0.308	0.938	0.336	0.928	

CD: Chamfer-L1 in % of the bbox diagonal · F@1%: F-score at 1% of the diagonal · NC: normal consistency · WT: watertight (VGGSfM mesh) · * beyond the course list

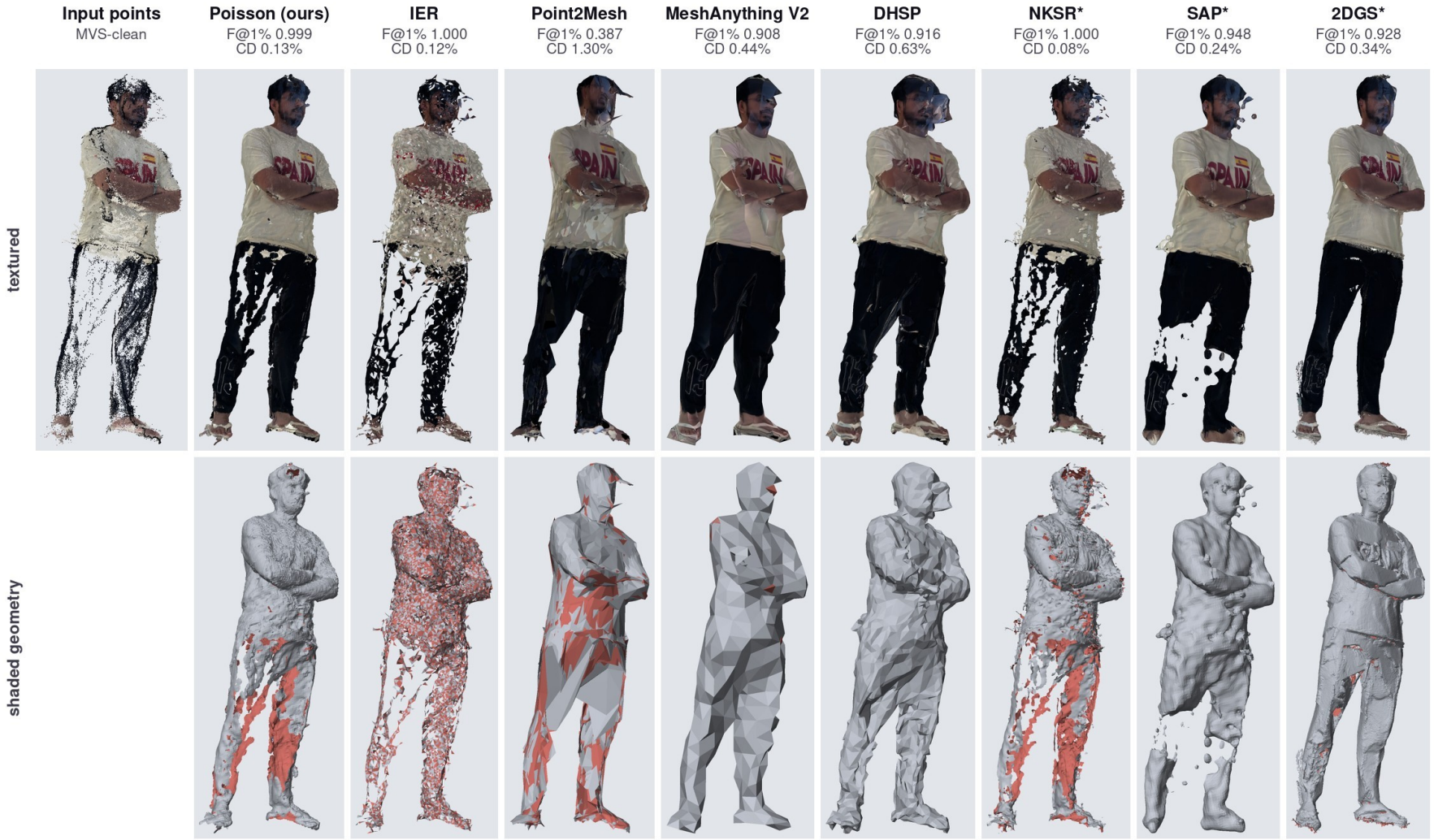
Qualitative comparison (VGGSfM)

Top: textured · bottom: shaded geometry · red = back-facing (inconsistently oriented) faces



Denser but noisier: the MVS clouds

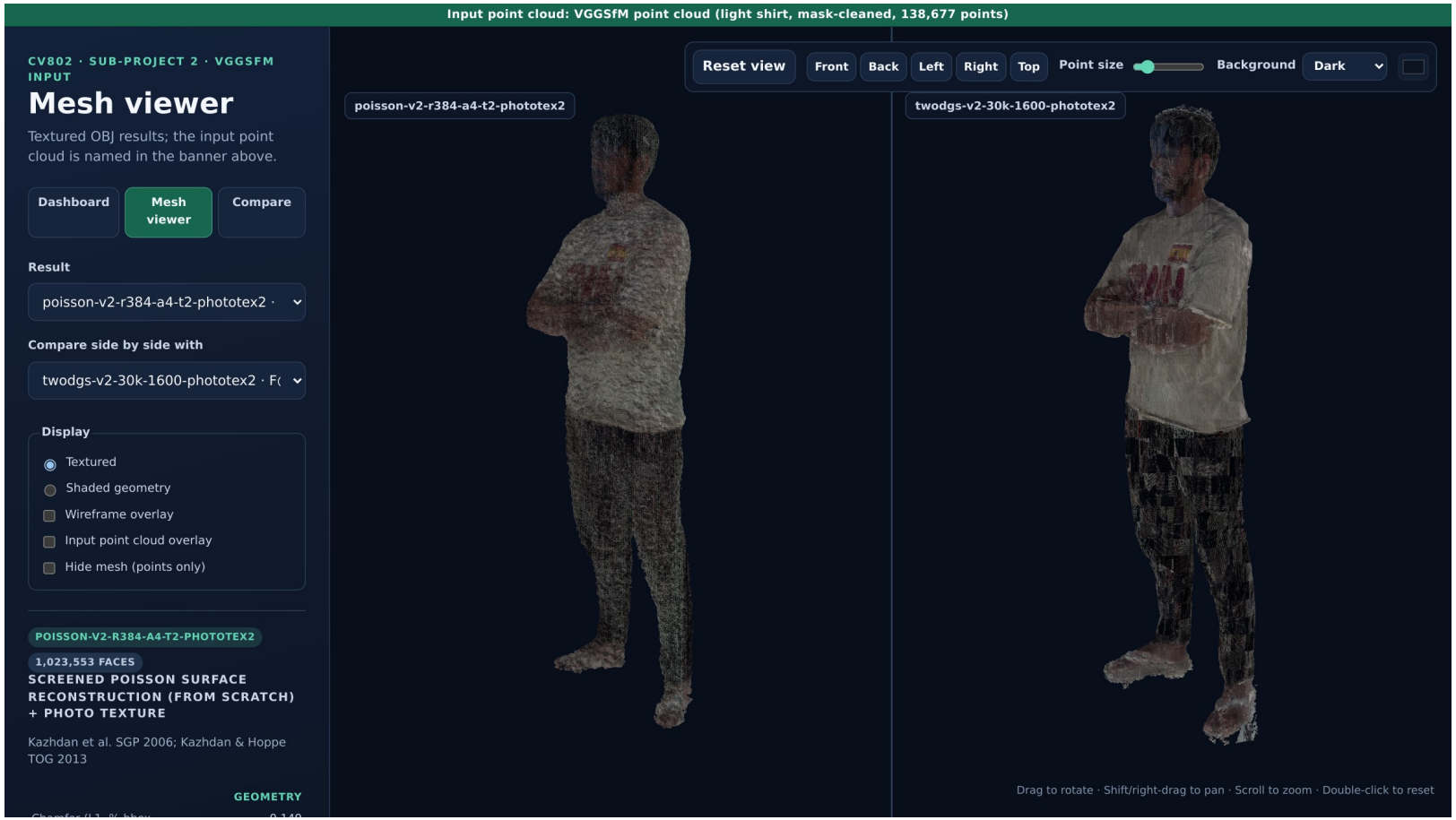
MVS-clean (404 floaters removed)

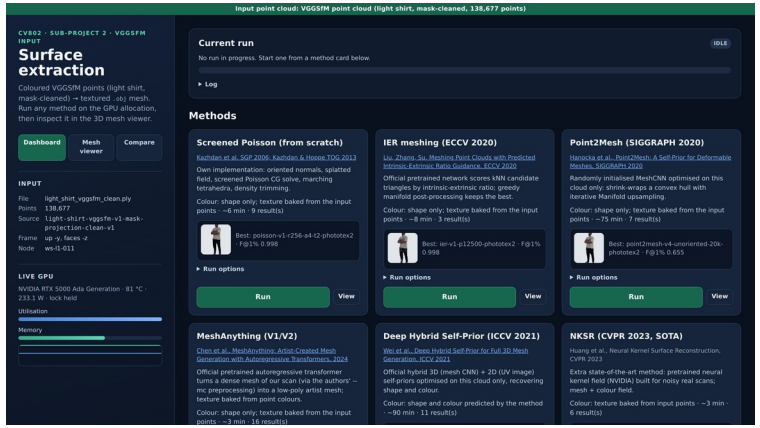


- Denser input → lower CD for point-following methods (NKSR 0.08%)
- Dark trousers: few MVS points → holes in open meshes
- Outliers: SAP blobs, Point2Mesh membranes (raw MVS F 0.13 → clean 0.39)
- VGGSfM stays the better input for closed, printable meshes

Bonus: the mesh viewer

Dependency-free Python server + three.js; runs the methods and compares results



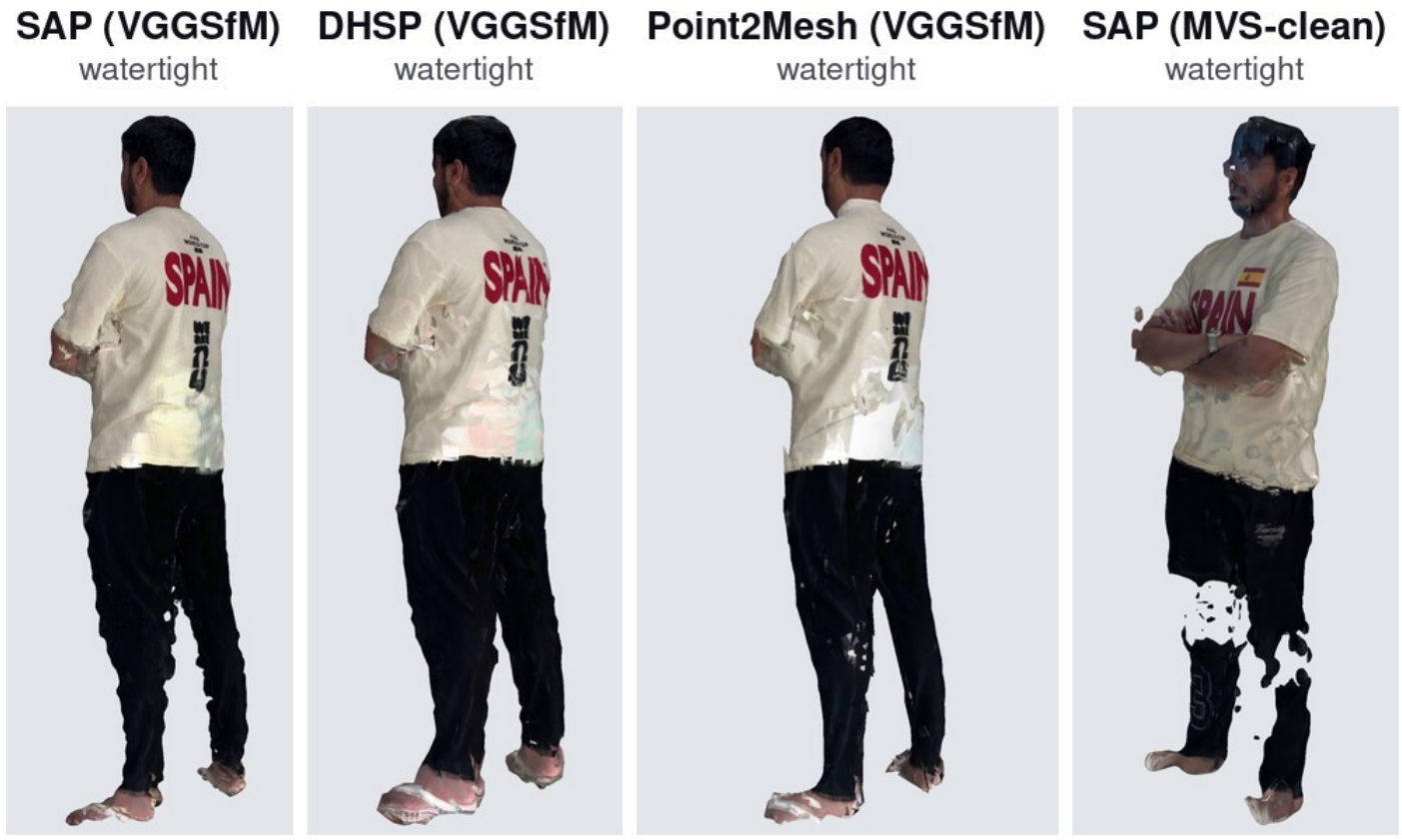


- textured / shaded / wireframe / input-point overlay
- two runs side by side, synchronised cameras
- dashboard: launch methods, progress, live GPU
- compare page: all runs ranked by metric

```
git clone …/cv802_assignment_2  &&  bash run_viewer.sh      (CPU only, no install)
```

3D printing

print_export.py: largest component, outward normals, +Z up, feet at z = 0, 180 mm tall



Mesh (VGGSfM)	Watertight	Volume	Printable
Poisson watertight	yes	101 cm³	yes
SAP	yes	101 cm³	yes
DHSP (full)	yes	126 cm³	yes
Point2Mesh	yes	97 cm³	yes
NKSR	no	-	no
2DGS	no	-	no

- STL (single colour) + OBJ/texture (colour printers)
- Recommended: SAP or DHSP (clean, closed); Poisson watertight for most detail
- Not checked: self-intersections; supports under crossed arms

What we learned

1

A careful classical method is competitive
our from-scratch Poisson ties the pretrained networks on VGGSfM (F@1% 0.999, 39 s)

2

Point metrics hide surface quality
DHSP's crumpled prior scored F@1% 0.999; check NC and shaded renders

3

Learned methods live in their training domain
IER: point density · MeshAnything: mesh-like input (0.51 → 0.987) · NKSR: metric scale

4

Self-priors are fragile on real scans
Point2Mesh: normal orientation (+0.20 F) and outliers; DHSP: re-mesh before OptCuts

5

Texture comes from the photos
visibility-aware projection makes the lettering readable; points alone blur

Deliverables and code

Week-5 slide 8: presentation · code · one-page document

github.com/anas2908/cv802_assignment_2

- run_viewer.sh — one command, CPU only
- results/ — best mesh of every method × 3 inputs (zipped)
- surface/methods/<method>/ — setup.sh, run.sh, pipeline.json
- surface/common/ — normals, texture, photo texturing, metrics, renderer
- surface/scripts/ — queue, finalize, print export, packaging
- surface/ui/ — server, dashboard, viewer, compare
- surface/docs/ — one-page report, slides, result tables

Deliverable	Where
Textured .obj for every method × input	results/*.zip, viewer
Code (all methods + training)	surface/
One-page report (CVPR style)	surface/docs/deliverables/report
Presentation	surface/docs/deliverables/slides
Mesh viewer (bonus)	run_viewer.sh
3D-print files	print/ in each watertight run

Team and contributions

Week-5 slide 9 asks for job assignments

Member	Responsibility
[name]	screened Poisson from scratch: normals, solver, marching tetrahedra
[name]	learned methods: IER, MeshAnything, NKSR; environments
[name]	self-prior methods: Point2Mesh, DHSP (re-mesh + OptCuts pipeline), SAP, 2DGS
[name]	photo texturing, metrics, mesh viewer, 3D-print export, report and slides

Thank you — questions?

Backup: metric definitions

All computed against the input cloud P; mesh M sampled with 200k points

$$\mathrm{CD} = \frac{1}{2}\left(\frac{1}{|M|}\sum_{m\in M}\min_{p\in P}\|m-p\| + \frac{1}{|P|}\sum_{p\in P}\min_{m\in M}\|p-m\|\right)$$

$$F_\tau = \frac{2\,\mathrm{Pr}\,\mathrm{Re}}{\mathrm{Pr}+\mathrm{Re}}, \qquad \mathrm{Pr} = \frac{|\{m : d(m,P) < \tau\}|}{|M|}, \qquad \mathrm{Re} = \frac{|\{p : d(p,M) < \tau\}|}{|P|}$$

$$\mathrm{NC} = \frac{1}{|M|}\sum_{m}|\mathbf{n}_{\mathrm{face}}(m)\cdot\mathbf{n}_P(\mathrm{nn}(m))|$$

- τ = 1% (and 0.5%) of the bounding-box diagonal; CD reported as % of the diagonal
- PSNR: texture sampled at the closest surface point of each input point vs that point's RGB
- Limitation: no ground-truth surface — metrics measure agreement with a noisy input, not truth

Backup: runtime and memory (VGGSfM)

One RTX 5000 Ada, 32 GB

Method	Run	Faces	Method time	Peak GPU
Poisson (ours)	poisson-v2-r384-a4-t2	1.02M	39s	3.6 GB
IER	ier-v1-p12500	17k	29s	-
Point2Mesh	point2mesh-v3-paper-unoriented	10k	42min	-
MeshAnything V2	meshanything-v2-mesh-mc7-greedy	1.4k	77s	-
DHSP	dhsp-v5-lap1-remesh-optcuts	6.0k	101min	-
NKSR	nksr-v2-ours-d1-mise1	2.09M	7s	2.9 GB
SAP	sap-v1-p100k-g256	64k	8min	-
2DGS	twodgs-v2-30k-1600	657k	47min	-

DHSP: 3D prior ≈ 100 min + 2D pipeline ≈ 35 min; photo texturing adds 1-5 min per mesh

Backup: how the code is organised

`run_viewer.sh`	one-command CPU viewer
`results/`	packaged meshes per input (zip)
`surface/inputs/`	the three input clouds
`surface/common/`	io, knn, normals (PCA+MST), texture (xatlas bake), phototex (photo projection), raster, render, metrics, finalize
`surface/methods/poisson/`	poisson.py (our solver + marching tetrahedra)
`surface/methods/<m>/`	setup.sh · run.sh · pipeline.json for ier, point2mesh, meshanything, dhsp (+ run_2d.sh), nksr, sap, twodgs
`surface/scripts/`	node_run, queue, best_queue, extra_queue, finalize, select_best, print_export, clean_cloud, package_results
`surface/ui/`	server.py + web/ (dashboard, viewer, compare)
`surface/docs/deliverables/`	results.json → report, slides, figures